Marcel McNeese Jr.
UH ID: 2274175
Due: Oct 29, 2026

$oftware ¢orp

Smart Parking Platform

Existing Software Landscape

Real time parking platforms:

1. Park Houston - allows for reservation of street parking
2. Park Mobile - allows payment of street parking in certain zones
3. Downtown Houston Website - Allows for parking reservations in lots
4. Ideal Parking Management - Management service that provides lot operators with tools to control their parking lot

How our application will be different

The applications listed above are great, but do not offer the solution that many are looking for.Our application will have the ability to estimate parking spot capacity for street parking, as that can be variable, but will display exact lot/ garage capacity, for easier parking. The others also do not include integrated directions for guidance to spots, giving us a competitive edge that makes our application more modern comparatively.

SRS Draft

• Software Requirements Specification (SRS)
• Project: Smart Parking Platform
• Version: 5.0
• Date: 10.8.2026

• 1. Introduction

This Smart Parking Platform (S.P.P.) will be brought to you by $oftware ¢orp. We are a development studio aimed at delivering solutions to real world problems, by making lives easier one app at a time. The project to create a S.P.P. was acquired through a close partnership with the City of Houston to create a solution to the parking frustrations Houstonians face.

• 1.1 Purpose

• This document defines the requirements for the Smart Parking Platform

• 1.2 Scope

• In scope is to create an interactive application that will allow users to view the status of parking spaces in real time. For operators of parking spots, this will streamline parking spot payment and tracking.

• Out of scope will be any type of physical hardware such as parking gates and cameras and general street parking.

• 2. Overall Description

• Users: Drivers and Vehicle Owners, Parking Lot Operators, Payment Services, Finance and Billing Services, City parking departments, System Admins

• The application will have the ability to be run on Mobile, and the Web.

• The mobile devices will need to have access to the App Store for ios devices, and the Google Play Store for Android devices.

• Assumptions: Internet Access, modern mobile device or computer, current software version of user device for app compatibility

3. Functional Requirements

• FR1: The system shall allow users to register and log in securely using email or Google/Apple authentication.
• FR2: The system shall show current availability of parking spaces within an interactive map.
• FR3: The system shall allow users to reserve parking spaces for their desired amount of time.
• FR4: The system shall handle digital payments, and give the user a receipt for proof of purchase.
• FR5: The system shall send notifications to let the user know their parking reservation will expire soon.
• FR6: The system shall give parking operators a view of current capacity and money generated in their desired timeframe.
• FR7: The system shall allow operators to adjust prices of parking spots at their discretion.
• FR8: The system shall link with third party navigation systems, such as Google Maps, to give directions to the space.

• 4. Non-Functional Requirements

• NFR1: The system shall support at least 1,000 concurrent users.
• NFR2: The system shall update parking spot availability within 1 second of a change in occupancy.
• NFR3: The system shall provide 99.99% uptime.
• NFR4: The system shall process payments within 5 seconds.

• 5. Use Case Examples

5.1
Title: Register a new user account
Actor: Driver
Precondition: User has downloaded the app and has a valid email address
Steps:

1. User selects "Sign Up"
2. User enters email, password, and vehicle information
3. System sends a verification email
4. User confirms email and account is activated
   Postcondition: User can log in and access the platform

5.2
Title: Reserve a parking space
Actor: Driver
Precondition: User is logged in and has a valid payment method on file
Steps:

1. User opens the map and selects an available parking space
2. User selects a reservation time window
3. User confirms payment
4. System reserves the space and sends a confirmation with directions
   Postcondition: Space is marked unavailable to other users for the reserved window

5.3
Title: Cancel a reservation
Actor: Driver
Precondition: User has an active reservation
Steps:

1. User navigates to "My Reservations"
2. User selects the reservation to cancel
3. User confirms cancellation
4. System processes any applicable refund per cancellation policy
   Postcondition: Space is released and marked available for other users

5.4
Title: View real time parking availability
Actor: Driver
Precondition: User has location services enabled and internet access
Steps:

1. User opens the app
2. System retrieves current location
3. System displays nearby parking facilities on an interactive map
4. Map updates availability in real time
   Postcondition: User can view up to date open spaces near them

5.5
Title: Get navigation directions to a reserved space
Actor: Driver

Precondition: User has an active reservation
Steps:

1. User selects "Navigate" from reservation details
2. System pulls route data from the mapping service
3. System displays turn by turn directions
4. User follows route to the facility
   Postcondition: User arrives at the correct facility and space

5.6
Title: Process a digital payment
Actor: Driver
Precondition: User has confirmed a reservation and has a saved payment method
Steps:

1. System calculates total cost based on time and facility rate
2. User confirms the charge
3. System sends transaction to third party payment provider
4. Payment provider returns confirmation
   Postcondition: Transaction is completed and a digital receipt is issued

5.7
Title: View reservation history and receipts
Actor: Driver
Precondition: User is logged in and has at least one past reservation
Steps:

1. User navigates to "History"
2. System displays list of past reservations
3. User selects a specific reservation
4. System displays receipt details
   Postcondition: User can view or export a past transaction record

5.8
Title: Receive a reservation expiration notification
Actor: Driver

Precondition: User has an active reservation nearing its end time
Steps:

1. System monitors time remaining on active reservations
2. System detects reservation is within 15 minutes of expiring
3. System sends a push notification to the user
4. User can choose to extend or end the reservation
   Postcondition: User is alerted before losing the reserved space

5.9
Title: Extend an active reservation
Actor: Driver
Precondition: User has an active reservation and the space has no conflicting reservation
Steps:

1. User selects "Extend Reservation"
2. System checks space availability for the extended window
3. User confirms additional payment
4. System updates the reservation end time
   Postcondition: Reservation is extended and space remains held

5.10
Title: Register a parking facility (operator onboarding)
Actor: Parking Facility Operator
Precondition: Operator has a valid business account request
Steps:

1. Operator submits facility details and documentation
2. System routes request to administrative approval queue
3. Administrator reviews and approves the facility
4. System activates the facility and its spaces in the platform
   Postcondition: Facility appears in the app for drivers to reserve

5.11
Title: Adjust pricing based on demand
Actor: Parking Facility Operator

Precondition: Operator is logged into the administrative dashboard
Steps:

1. Operator navigates to "Pricing Settings"
2. Operator selects a time block or demand condition
3. Operator sets a new rate
4. System applies the updated rate to future reservations
   Postcondition: New pricing is reflected for drivers making reservations

5.12
Title: Monitor real time occupancy
Actor: Parking Facility Operator
Precondition: Operator is logged into the administrative dashboard
Steps:

1. Operator opens the "Occupancy" view
2. System displays current occupied vs. available spaces
3. Operator filters by zone or level
4. Dashboard updates counts in real time
   Postcondition: Operator has an accurate live view of facility usage

5.13
Title: Generate an occupancy and revenue report
Actor: Parking Facility Operator
Precondition: Operator is logged into the administrative dashboard and facility has reservation history
Steps:

1. Operator selects "Reports"
2. Operator chooses a date range and report type
3. System compiles occupancy and revenue data
4. System generates a downloadable report
   Postcondition: Operator receives a report for business analysis
1. Billing staff reviews the dispute ticket
2. Staff cross references transaction and reservation logs
3. Staff determines validity of the dispute
4. Staff issues a refund or denies the claim with explanation
   Postcondition: Dispute is resolved and user is notified of the outcome

5.15
Title: Suspend a facility for policy violation
Actor: System Administrator

Precondition: Administrator has identified a facility violating platform policy
Steps:

1. Administrator opens the facility's account in the admin panel
2. Administrator reviews violation details
3. Administrator selects "Suspend Facility"
4. System disables the facility's listings and notifies the operator
   Postcondition: Facility no longer appears to drivers until issue is resolved

WorkBreakdownStructure

1. Requirements & Planning

- 1.1 Competitive & Market Research
- 1.2 Vision & Scope Definition
- 1.3 Software Requirements Specification
- 1.4 Requirements Validation & Stakeholder Approval
    - 1.4.1 Review requirements
    - 1.4.2 Present SRS draft to stakeholders
    - 1.4.3 Obtain formal approval

2. Design

- 2.1 System Architecture Design
- 2.2 UI/UX Design
- 2.3 Database & API Design
- 2.4 Design Review & Validation
    - 2.4.1 Conduct architecture walkthrough
    - 2.4.2 Conduct UI/UX design review
    - 2.4.3 Validate API contracts against requirements

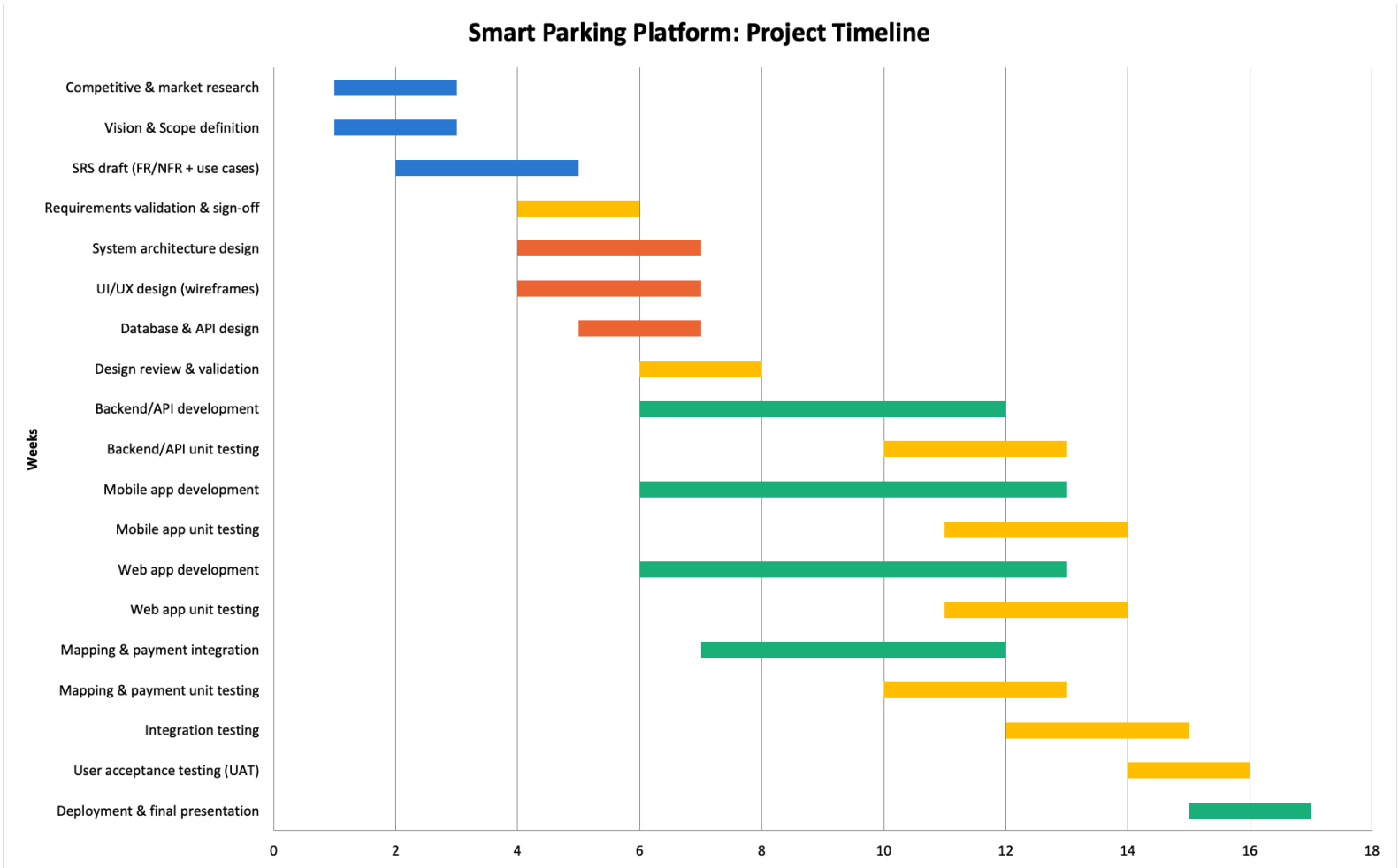
3 Development

- 3.1 Backend/API Development
    - 3.1.1 Build authentication service
    - 3.1.2 Build reservation & availability service
    - 3.1.3 Unit test backend services
- 3.2 Mobile App Development
    - 3.2.1 Build user registration and login flow
    - 3.2.2 Build map & reservation UI
    - 3.2.3 Unit test mobile app modules
- 3.3 Web App Development
    - 3.3.1 Build operator administrative dashboard
    - 3.3.2 Build pricing management interface
    - 3.3.3 Test web app
- 1.3.4 Mapping & Payment Integration
    - 3.4.1 Integrate mapping/navigation API
    - 3.4.2 Integrate payment provider
    - 3.4.3 Unit test integration endpoints

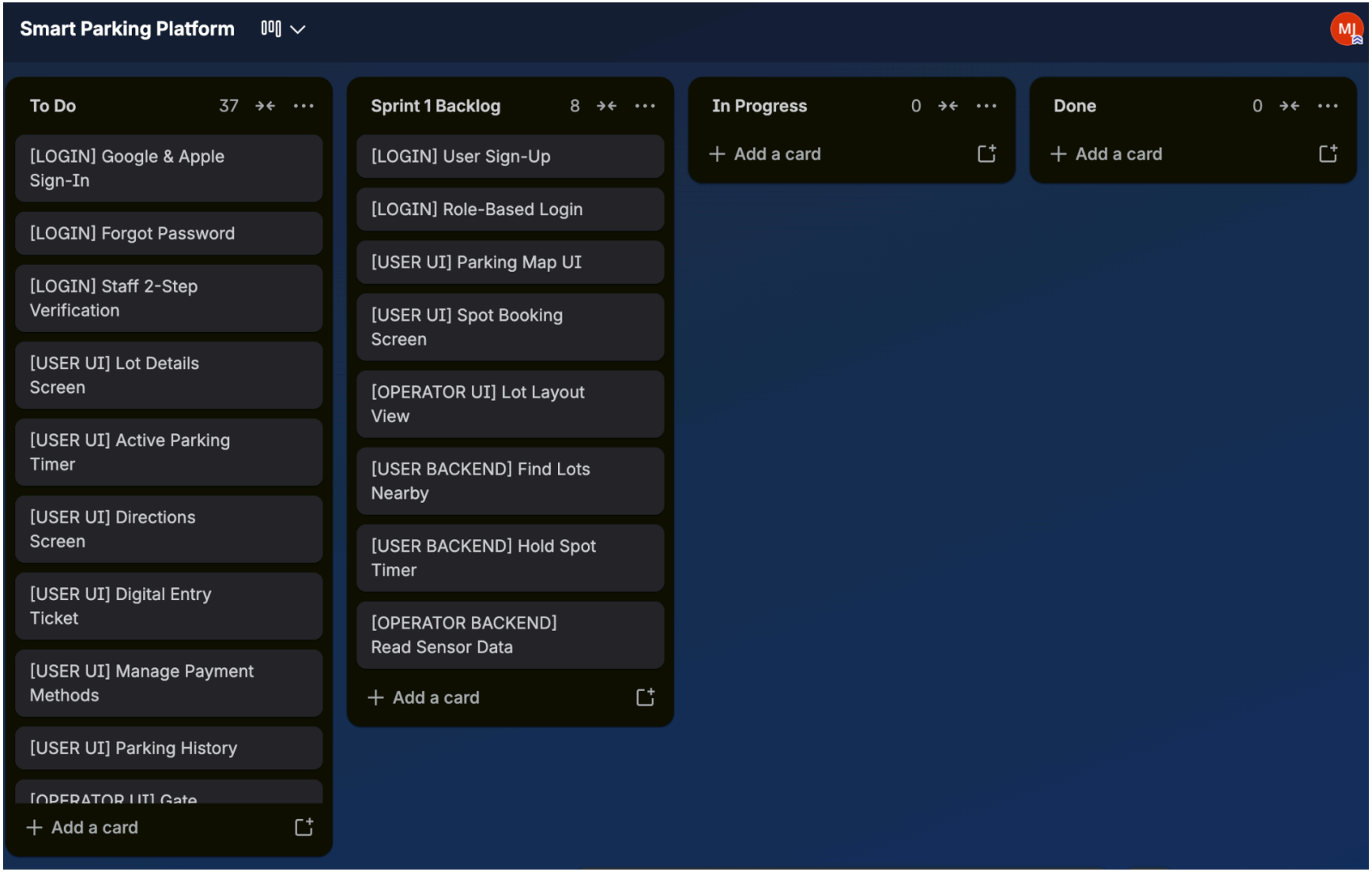
4 Integration, UAT & Deployment

- 4.1 Integration Testing
    - 4.1.1 Test cross-service data flow
    - 4.1.2 Bug triage & fixes
- 4.2 User Acceptance Testing
    - 4.2.1 Conduct UAT sessions with stakeholders
    - 4.2.2 Collect and prioritize feedback
- 4.3 Deployment & Final Presentation
    - 4.3.1 Prepare deployment/launch plan
    - 4.3.2 Finalize project documentation
    - 4.3.3 Deliver final presentation

GANNT CHART



Smart Parking Platform Scrum Board



Smart Parking Platform

Identifying risks

Technical	Schedule	Financial	People
Parking sensors not displaying correct information for operator	Missing deployment deadline	Exceeding budget	Team member having an unexpected reason to stop working
Unexpected bugs when released	Components of UI development taking longer than expected	Project contract being cancelled	Team member quitting the project
User parking lot map not displaying correctly on different user devices	Features of UX not being ready by deployment	New hardware needed being too expensive for lot operators	Team member underperforming on deliverables
Reservation ability not working properly leading to overlaps in reserved times		Being beat to the market by a parking application competing in the same market	Team conflicts causing a delay in deliverables

Risk Register

ID	Description	Probability	Impact	Owner	Response Strategy	Status/Notes
1	Missing deployment deadline	3	5	Project Manager	Mitigate: build schedule buffer into the Gantt chart; track milestones weekly against plan	Open - monitoring
2	Exceeding budget	2	4	Finance Lead	Mitigate: track spend weekly against budget; flag overruns early for stakeholder review	Open
3	Unexpected bugs when released	4	4	QA Lead	Mitigate: expand UAT coverage before deployment; staged rollout instead of full launch	Open
4	Reservation ability not working properly, leading to overlaps in reserved times	3	5	Backend Lead	Mitigate: implement space-locking logic with automated conflict-detection testing	Open
5	Team member having an unexpected reason to stop working / quitting the project	2	4	Project Manager	Mitigate: maintain updated documentation/handoff notes; identify backup owners for key modules	Monitoring
6	Being beat to market by a competing parking application	3	4	Product/Marketing	Mitigate: accelerate MVP timeline; emphasize differentiation (exact lot capacity + navigation)	Monitoring
7	Parking sensors not displaying correct information for operator	3	3	Mapping/Hardware	Mitigate: implement sensor data validation checks with manual override fallback for operators	Open
8	Project contract being cancelled	1	5	Project Manager	Mitigate: maintain clear, frequent stakeholder communication and documented scope agreements	Monitoring
9	Team conflicts causing a delay in deliverables	2	3	Project Manager	Mitigate: establish clear roles/responsibilities and a conflict-resolution process upfront	Open
10	Components of UI development taking longer than expected	3	3	UI/UX Lead	Mitigate: break UI work into smaller sprints with clear estimates; reprioritize scope if needed	Open

Communication Plan

Team	**Cadence**	**Method**
Full Dev Team	Daily Standup	Microsoft Teams
Full Dev Team	Sprint planning and review	Teams and Trello
Team Leads	Weekly meeting	Slack
QA and Testing	Daily Bug status	Slack and Trello
Stakeholders	Weekly meeting	Teams

1. Roles and Resources

Role	Name	Skill Level	Skill Assumptions	Allocation (FTE%)
Project Manager	Lamar	Senior	8+ yrs: Agile/Scrum, manages schedule, budget, risk, and stakeholders	100%
Mobile App Developer	Sophie	Senior	7+ yrs: iOS/Android development, app store release experience	100%
Web App Developer	Abraham	Mid	4 yrs: Front end, operator dashboards	100%
Backend Developer	Josh	Senior	8+ yrs: API design, databases, AWS cloud	100%
Mapping & Location Services Developer	Marcel	Senior	6+ yrs: world data, Google Maps Platform, real-time location	50%
Payment Services Developer	Alexis	Senior	7+ yrs: payment gateways, security	50%
Tester	Syreeta	Mid	4 yrs: test automation, integration testing	70%
UI/UX Designer	Jake	Mid	5 yrs: usability testing, accessibility	50%

2. Resource and Cost Plan

Item	Category	Qty/Months	Unit Cost	Duration(months)	Total	Notes
Team Labor	Labor	1	$80,720	4	$322,880	Amount of labor per month for the 4 months
Cloud hosting (AWS)	Infrastructure	1	$2,500	4	$10,000	Monthly: servers, database etc.
Google Maps Platform API	Third-party services	1	$1,200	4	$4,800	Monthly: map display, geocoding, navigation directions
Payment provider setup	Compliance	1	$8,000	1	$8,000	One-time: gateway onboarding and security compliance check
App store developer accounts	Licenses	1	$124	1	$124	One-time: Apple $99/yr + Google Play $25 registration
Test devices (iOS/Android phones & tablets)	Hardware	6	$600	1	$3,600	One-time: cross-device testing for map/UI display
Collaboration tools (Trello, Slack, Teams)	Software licenses	8	$25	4	$800	Per user per month, 8 team members
Third-party security & penetration testing	Services	1	$6,000	1	$6,000	One-time: external security assessment before launch
Total					$356,204	

3. RACI Matrix

Task / Deliverable	Sponsor (City of Houston)	Project Manager	UI/UX Designer	Backend/ API Dev	Web App Dev	Mobile App Dev	Mapping Dev	Payment Dev	QA Engineer
Deployment & final presentation	I	A/R	I	R	R	R	C	C	C
User acceptance testing	C	A	C	I	I	I	I	I	R
Integration testing	I	I	I	C	C	C	C	C	A/R
Mapping & payment integration	I	I	I	A	C	C	R	R	C
Web app development	I	I	C	C	A/R	I	C	C	C
Mobile app development	I	I	C	C	I	A/R	C	C	C
Backend/API development	I	I	I	A/R	C	C	C	C	C
Design review & validation	C	A	R	R	C	C	C	C	C
Database & API design	I	I	I	A/R	C	C	C	C	I
UI/UX design (wireframes)	C	I	A/R	C	C	C	I	I	C
System architecture design	I	C	C	A/R	C	C	C	C	I
Requirements validation & sign-off	A	R	C	C	C	C	C	C	R
SRS draft	C	A/R	C	C	C	C	C	C	C
Vision & Scope definition	C	A/R	I	I	I	I	I	I	I
Competitive & market research	I	A	R	I	I	I	I	I	I
Weekly status reporting & stakeholder updates	I	A/R	I	I	I	I	I	I	I
Risk & budget management	C	A/R	I	C	C	C	C	C	C

LEGEND:

R = Responsible

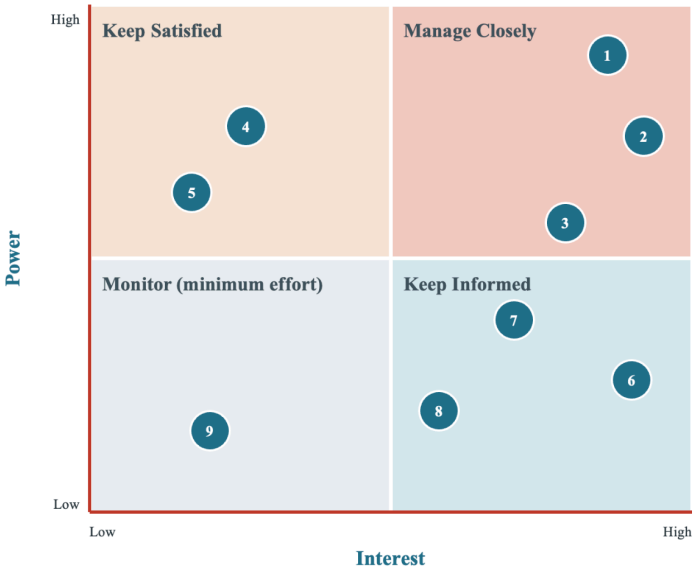
A = Accountable

A/R = Accountable and Responsible

C = Consulted

I = Informed

Stakeholder Power Interest Grid: Smart Parking Platform



#	Stakeholder	Strategy
1	City of Houston (sponsor / transportation dept.)	*Manage closely*
2	Parking facility operators	*Manage closely*
3	Dev team	*Manage closely*
4	Third-party payment providers	*Keep satisfied*
5	Mapping and navigation APIs	*Keep satisfied*
6	Drivers & mobile app users	*Keep informed*
7	System admins	*Keep informed*
8	Finance & billing teams	*Keep informed*
9	General public (non-users)	*Monitor*

PROJECT CHARTER

Project Title	Smart Parking Platform	**Project Manager**	Marcel McNeese Jr.
Project Start	Week 1	**Project End**	Week 16
Project Sponsor(s)	City of Houston, Transportation Department	**Delivery Company**	$oftware ¢orp

Business Need

Drivers in Houston lose time and patience waiting for parking, which adds greatly to city congestion and below satisfactory happiness levels. Parking operators lack real-time tools to monitor occupancy, manage pricing, and improve utilization. This project delivers a web and mobile platform that shows real-time availability, lets drivers reserve and pay digitally, and gives operators a dashboard with occupancy reporting and analytics.

Project Scope

- Real-time availability and interactive map for garages and lots
- Reservations, digital payments, receipts, notifications
- Operator dashboard, pricing tools, occupancy reporting
- Web and mobile (iOS/Android) applications

Deliverables

- Software Requirements Specification
- Web application and mobile applications
- Backend/API services with mapping and payment integration
- Operator admin dashboard and reports
- Test results
- Final documentation and presentation

Risks & Issues

- Missing the deployment deadline
- Unexpected bugs at release
- Reservation overlaps / double booking
- Budget overrun

Assumptions / Dependencies

- Third-party mapping and payment APIs remain available at current terms
- Operators provide occupancy data
- The City of Houston provides all compliance requirements
- Users have internet access whenever they use the app

Financials

Total budget: $356,204

- Labor: $322,880
- Non-labor: $33,324 (cloud, Maps API, compliance, tools, testing)
- Spend of about $90,000 over 4 months

Success Criteria

- Availability updates within 1 second of an occupancy change
- Supports at least 1,000 concurrent users
- 99.99% uptime
- Payments processed within 5 seconds

Milestones Schedule

Milestone	Target Completion	Owner
Requirements validated and signed off	End of Week 5	Project Manager
Design reviewed and approved	End of Week 7	Project Manager
Development complete and unit tested	End of Week 13	Development leads
Integration testing complete	End of Week 14	QA Engineer
UAT accepted by sponsor	End of Week 15	Project Manager
Deployment and final presentation	End of Week 16	Project Manager

Approvals

Role	Name	Signature	Date
Sponsor	City of Houston Representative		
Project Manager	Marcel McNeese Jr.		

TEAM CHARTER

Smart Parking Platform | $oftware ₡orp | Version 1.0

Mission

Deliver a smart parking platform that ends the endless search for a spot in Houston and gives lot operators real-time control, by working as one open, accountable team.

Scope

Within: requirements, design, development, testing, and launch of the web and mobile platform for garages and lots.

Outside: physical hardware (sensors, gates, cameras), street parking, and post-launch operations.

Goals

- Deliver in 16 weeks and within the $356,204 budget
- Zero double-booked reservations at launch
- Pass stakeholder with no critical defects
- Keep every stakeholder informed weekly

Roles & Responsibilities

Name	Role	Responsibilities
Marcel	Project Manager	Plan, budget, risk, stakeholder updates; runs Scrum
Sophie	Mobile App Developer	iOS/Android app, map and reservation UI, notifications
Abraham	Web App Developer	Operator dashboard, pricing and reporting views
Josh	Backend Developer	Authentication, reservation and availability services, database
Lamar	Mapping & Location Developer	Map display, navigation, real-time location data
Alexis	Payment Services Developer	Payment gateway, receipts, payment security
Syreeta	Tester	Test plans, automation, integration testing
Jake	UI/UX Designer	Wireframes, design reviews, usability testing

Strengths & Skills

- Senior depth in mobile, backend, mapping, and payments (6-8+ years)
- Mid-level web, QA automation, and UX talent
- Agile project management and budget tracking
- A balanced mix of leaders, analysts, implementers, and communicators

Metrics for Success

- 100% of the 6 charter milestones met on time
- Budget variance within +/-5%
- At least 85% of sprint commitments met
- 0 open critical defects at launch
- Targets met: 1-second availability updates, 99.99% uptime

Values

- **Transparency:** raise blockers early
- **Ownership:** finish what we commit to
- **Quality:** test as we build
- **Respect:** disagree openly, decide together

Norms

- 15-minute daily standup; Slack replies within 4 business hours
- Decisions are democratic by default; the PM decides quickly during major problems
- Conflicts: talk directly first and solve together; PM mediates if unresolved
- Sprint review and retrospective every sprint

Communications Plan & Coordination Tools

Meeting / Channel	Cadence	Tool
Full Dev Team standup	Daily	Microsoft Teams
Sprint planning, review, and retrospective	Each sprint	Teams + Trello
Team leads sync	Weekly	Teams
Sub-team syncs (mobile, web, backend, mapping, payment)	Weekly	Slack
QA and Testing	Daily	Slack + Trello
Stakeholder progress update	Weekly	Recorded video (MS Streams) + GitHub
Code and documents	Continuous	GitHub

INTERNAL: for $oftware ¢orp use only.

Project Closure Report

Smart Parking Platform

Project	Smart Parking Platform
Project Manager	Marcel McNeese
Sponsor	City of Houston, Transportation Department
Version / Date	1.0, November 2026

1. Executive Summary

The Smart Parking Platform project set out to produce a complete project management plan for a web and mobile system that helps Houston drivers find, reserve, and pay for parking in real time, while giving garage operators tools to monitor occupancy, set pricing, and a dashboard that reports on usage.

No software was built, so the plan's performance targets and cost estimates remain untested. The main shortfalls are unvalidated cost and effort assumptions, a scope inconsistency around street parking, and the absence of direct stakeholder interviews. This report records what was achieved, what was learned, and what we would do differently on the next project.

2. Achievements and Shortfalls

Achievements

- Requirements baseline: an SRS with functional and nonfunctional requirements and 15+ use cases covering drivers, operators, billing staff, and administrators.
- Clear differentiation: a competitive analysis showing the gap our platform fills: exact lot/ garage capacity and integrated navigation.
- Testing: a Level 3 WBS and Gantt chart with reviews and unit testing inside each phase instead of one testing block at the end.
- Sprint backlog: 40+ items across login, UI, backend, and reporting in Trello, with an 8-item Sprint 1.
- Risk and cost control: a scored 10-item risk register, bottom-up cost plan with monthly cashflow, communication plan, and RACI matrix.
- Stakeholder management: a power interest analysis plus team and project charters that define roles, norms, milestones, and approvals.

Shortfalls

- Plan only: nothing was built or tested, so the performance targets are unproven.
- Unvalidated estimates: rates, allocations, and vendor costs are assumptions, and the team members are fictional.
- No direct stakeholder input: requirements came from the course brief rather than interviews or surveys with drivers, operators, or City staff.

4. Lessons Learned

Communication

- Measurable requirements (for example, “within 1 second”) gave every later document a shared target. Vague wording would have left room for disagreement.
- Budget, schedule, and role figures appear in several documents, so each change had to be carried through all of them. One master source would reduce errors.
- Different stakeholders need different update rhythms. The power interest grid showed who needs weekly updates and who needs milestone reviews.

Testing and quality

- Testing belongs in every phase. Requirement reviews, design reviews, and unit tests inside each phase catch problems earlier than a final test, a lesson learned with the post-mortems studied in class.
- A performance number in a requirement is only a hypothesis until it is measured, so risky targets should be prototyped early.

Planning and estimating

- A bottom-up estimate showed that labor is about 92% of the budget,

Team and closure

- Naming one accountable owner per deliverable in the RACI exposed shared work, such as integration across the backend, mapping, and payment developers.
- Closure is educational as well as administrative, so lessons are best captured while they are fresh.

5. Recommendations for Future Projects

1. Validate every cost and rate assumption with real vendor quotes and salary data before committing to a budget.
2. Interview or survey actual drivers, operators, and City staff to confirm requirements before design begins.
3. Decide scope boundaries up front, including street parking and how occupancy data reaches the platform, and record them in a single place.
4. Build an early prototype of the reservation and availability engine to test the 1-second and double-booking targets.
5. Convert the week-based schedule to calendar dates, add a quantified contingency, and review it every sprint.
6. Keep a single master source for figures that repeat across documents, trace each requirement to a test, and hold a short retrospective every sprint with results stored in a central lessons-learned log.

AI Usage Disclosure

AI Use Section	Reason
SRS Draft	AI was used in the section of Use Case examples, to help me brainstorm possible uses of the Smart Parking Platform. All use cases were verified, edited, and considered before adding to the document.
WBS Breakdown	Helped draft the section so I could get an understanding of how I should build it.
Communication Section	I used AI to help me understand what method of communication would be best for each group of people. This is because I did not know Slack would be best. I have heard of Slack, but have not personally used it.

X. Marcel McNeese Jr.